DATA STRUCTURE

SUBMITTED TO

SIR MUHAMMAD SABEEH

SUBMITTED BY

KHADIJA FAHID

25021598-0 93

COURSE CODE

SE 200

SECTION D



PROGRAM NO 1

ARRAY

```
importjava.utilScanner;
public class Question1Array {
public static voidmain(String[]args) {
Scannersc  = newScanner(System.in);
int[] arr= new int[10];
int size = 0;
int choice;
do {
System.out.println("\n----- ARRAY MENU -----");
System.out.println("1. Add value");
System.out.println("2. Insert at index");
System.out.println("3. Fill array");
System.out.println("4. Delete last element");
System.out.println("5. Delete by index");
System.out.println("6. Display");
System.out.println("7. Search value");
System.out.println("8. Get value at index");
System.out.println("9. Replace/Update value at index");
System.out.println("10. Size");
System.out.println("11. Exit");
System.out.print("Enter your choice: ");
choice =sc.nextInt();
switch (choice) {
case 1: {
if (size ==arr.length) {
System.out.println("Array is full.");
} else {
System.out.print("Enter value: ");
```

```
int value =sc.nextInt();
arr[size] = value;
size++;
System.out.println("Value added successfully.");
}
break;
}
case 2: {
if (size ==arr.length) {
System.out.println("Array is full.");
} else {
System.out.print("Enter index: ");
int index =sc.nextInt();
if (index < 0 ||index > size) {
System.out.println("Invalid index.");
} else {
System.out.print("Enter value: ");
int value =sc.nextInt();
for (int i = size; i > index;i--) {
arr[i] = arr[i - 1];
}
arr[index] = value;
size++;
System.out.println("Value inserted successfully.");
}
}
break;
}
case 3: {
if (size ==arr.length) {
System.out.println("Array is already full.");
```

```
} else {
System.out.println("Enter values.");
System.out.println("Enter -1 when you want to stop.");
while (size < arr.length) {
System.out.print("Enter value: ");
int value = sc.nextInt();
if (value == -1) {
break;
}
arr[size] = value;
size++;
}
System.out.println("Array filling completed.");
}
break;
}
case 4: {
if (size == 0) {
System.out.println("Array is empty.");
} else {
System.out.println("Deleted value: " + arr[size - 1]);
size--;
System.out.println("Last element deleted successfully.");
}
break;
}
case 5: {
if (size == 0) {
System.out.println("Array is empty.");
} else {
System.out.print("Enter index to delete: ");
```

```
int index = sc.nextInt();
if (index < 0 || index >= size) {
System.out.println("Invalid index.");
} else {
System.out.println("Deleted value: " + arr[index]);
for (int i = index; i < size - 1; i++) {
arr[i] = arr[i + 1];
}
size--;
System.out.println("Value deleted successfully.");
}
}
break;
}
case 6: {
if (size == 0) {
System.out.println("Array is empty.");
} else {
System.out.println("Array elements:");
for (int i = 0; i < size; i++) {
System.out.print(arr[i] + " ");
}
System.out.println();
}
break;
}
case 7: {
if (size == 0) {
System.out.println("Array is empty.");
} else {
System.out.print("Enter value to search: ");
```

```
int value = sc.nextInt();
boolean found = false;
for (int i = 0; i < size; i++) {
if (arr[i] == value) {
System.out.println(
"Value found at index: " + i
);
found = true;
}
}
if(!found) {
System.out.println("Value not found.");
}
}
break;
}
case 8: {
if (size == 0) {
System.out.println("Array is empty.");
} else {
System.out.print("Enter index: ");
int index = sc.nextInt();
if (index < 0 || index >= size) {
System.out.println("Invalid index.");
} else {
System.out.println(
"Value at index " + index + ": " + arr[index]
);
}
}
break;
```

```
}
case 9: {
if (size == 0) {
System.out.println("Array is empty.");
} else {
System.out.print("Enter index: ");
int index = sc.nextInt();
if (index < 0 || index >= size) {
System.out.println("Invalid index.");
} else {
System.out.print("Enter new value: ");
int value = sc.nextInt();
arr[index] = value;
System.out.println("Value updated successfully.");
}
}
break;
}
case 10: {
System.out.println("Current size of array: " + size);
break;
}
case 11: {
System.out.println("Program ended.");
break;
}

default: {
System.out.println("Invalid choice. Please try again.");
}
}
```

```
} while (choice != 11);
        sc.close ();
            }
           }
```

Output

```
----- ARRAY MENU -----
1. Add value
2. Insert at index
3. Fill array
4. Delete last element
5. Delete by index
6. Display
7. Search value
8. Get value at index
9. Replace/Update value at index
10. Size
11. Exit
Enter your choice:

5
Array is empty.

----- ARRAY MENU -----
```

PROGRAM NO 2

STACK

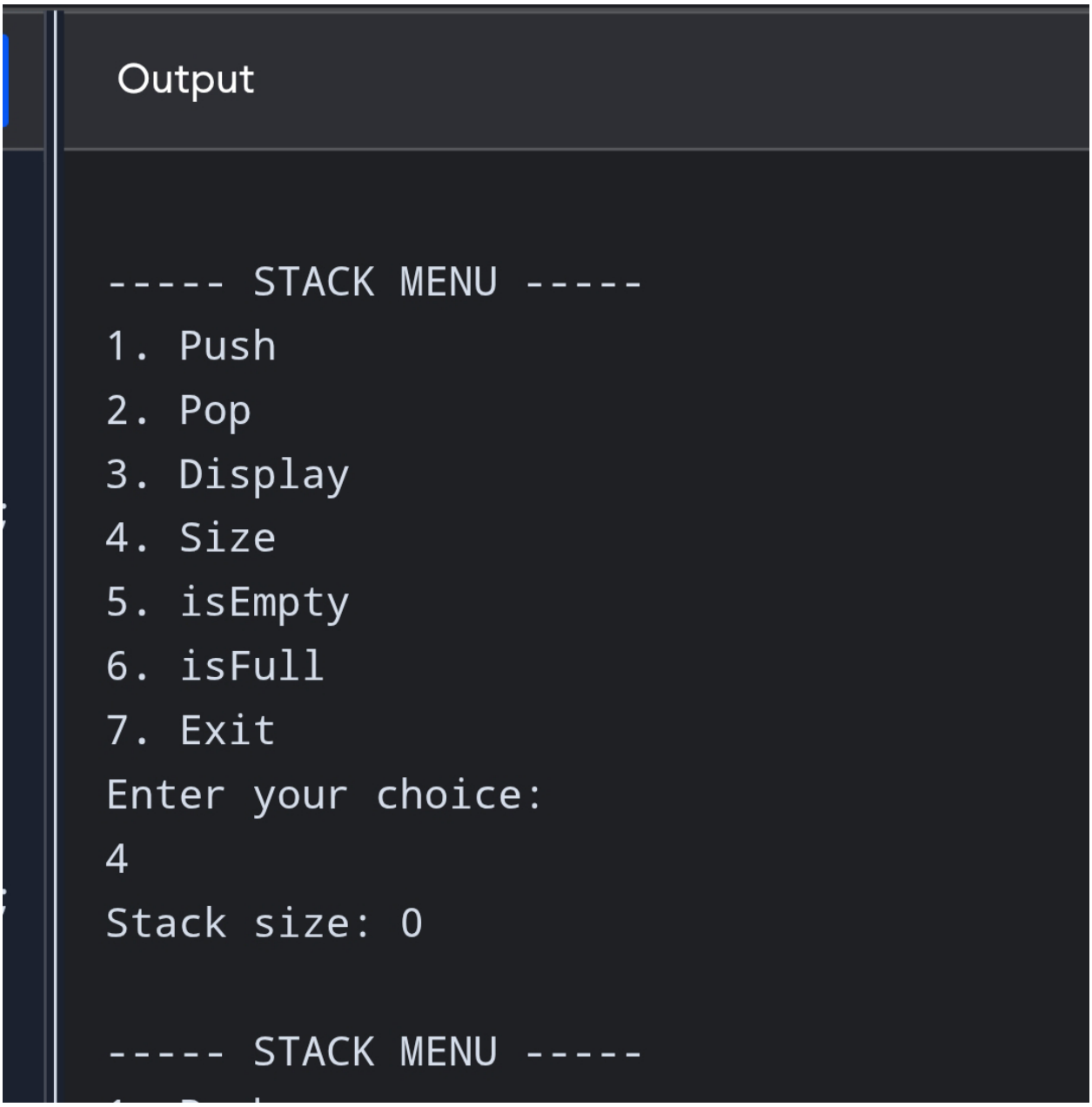
```
importjava.utilScanner;
public class Question2Stack {
static final int MAX = 10;
static int[] stack = new int[MAX];
static int top = -1;
public static void main(String[]args) {
 Scannersc  = newScanner(System.in);
int choice;
do {
System.out.println("\n----- STACK MENU -----");
System.out.println("1. Push");
System.out.println("2. Pop");
System.out.println("3. Display");
System.out.println("4. Size");
System.out.println("5. isEmpty");
System.out.println("6. isFull");
System.out.println("7. Exit");
System.out.print("Enter your choice: ");
choice = sc.nextInt();
switch (choice) {
case 1:
if (top == MAX - 1) {
System.out.println("Stack Overflow: Stack is full.");
} else {
```

```
System.out.print("Enter value: ");
int value = sc.nextInt();
top++;
stack[top] = value
System.out.println("Value pushed successfully.");
}
break;
case 2:
if (top == -1) {
System.out.println("Stack Underflow: Stack is empty.");
} else {
System.out.println("Popped value: " + stack[top]);
top--;
}
break;
case 3:
if (top == -1) {
System.out.println("Stack is empty.");
} else {
System.out.println("Stack values from top to bottom:");
for (int i = top; i >= 0; i--) {
System.out.println(stack[i]);
}
}
break;
case 4:
```

```
System.out.println("Stack size: " + (top + 1));
break;
case 5:
if (top == -1) {
System.out.println("Stack is empty.");
} else {
System.out.println("Stack is not empty.");
}
break;
case 6:
if (top == MAX - 1) {
System.out.println("Stack is full.");
} else {
System.out.println("Stack is not full.");
}
break;
case 7:
System.out.println("Program ended.");
break;
default:
System.out.println("Invalid choice.");
}

} while (choice != 7);
sc.close ();
}
```

}


Output
----- STACK MENU -----
1. Push
2. Pop
3. Display
4. Size
5. isEmpty
6. isFull
7. Exit
Enter your choice:
4
Stack size: 0
----- STACK MENU -----

PROGRAM NO 3

QUEUE

```
importjava.utilScanner;
public class Question3Queue {
public static voidmain(String[]args) {
Scannersc  = newScanner(System.in);
int[] queue = new int[10];
int front = 0;
int rear = -1;
int size = 0;
int choice;
do {
System.out.println("\n----- QUEUE MENU -----");
System.out.println("1. Enqueue");
System.out.println("2. Dequeue");
System.out.println("3. Display");
System.out.println("4. Size");
System.out.println("5. isEmpty");
System.out.println("6. isFull");
System.out.println("7. Exit");
System.out.print("Enter your choice: ");
choice = sc.nextInt();
switch (choice) {
case 1:
if (size ==queue.length) {
System.out.println("Queue is full. Cannot enqueue.");
```

```
} else {
System.out.print("Enter value: ");
int value = sc.nextInt();
rear++;
queue[rear] = value;
size++;
System.out.println("Value added successfully.");
}
Break;
case 2:
if (size == 0) {
System.out.println("Queue is empty. Cannot dequeue.");
} else {
System.out.println("Dequeued value: " + queue[front]);
for (int i = 0; i < size - 1; i++) {
queue[i] = queue[i + 1];
}
rear--;
size--;
if (size == 0) {
front = 0;
rear = -1;
}
System.out.println("Value removed successfully.");
}
break;
```

```
case 3:
if (size == 0) {
System.out.println("Queue is empty.");
} else {
System.out.println("Queue elements from front to rear:");
for (int i = front; i <= rear; i++) {
System.out.print(queue[i] + " ");
}
System.out.println();
}
break;
case 4:
System.out.println("Current queue size: " + size);
break;
case 5:
if (size == 0) {
System.out.println("Queue is empty.");
} else {
System.out.println("Queue is not empty.");
}
break;
case 6:
if (size == queue.length) {
System.out.println("Queue is full.");
} else {
System.out.println("Queue is not full.");
```

```
                }
                break;
            case 7:
                System.out.println("Program ended.");
                break;
            default:
                System.out.println("Invalid choice. Please try again.");
            }
        } while (choice != 7);
        sc.close ();
    }
}
```

Output

```
----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Display
4. Size
5. isEmpty
6. isFull
7. Exit
Enter your choice:
4
Current queue size: 0

----- QUEUE MENU -----
```

QUESTION 4

Write five real world examples and also write their advantages and disadvantages .

1. 1. Array

An arraystores elements in a fixed, ordered sequence and allows fast access using an index.

Real-world example	Why an array is a natural fit
1. Student marks	Marks for 30 students can be stored in an array, where eac position represents one student.
2. Monthly sales	The 12 monthly sales figures can be stored in an array, wit each index representing a month.
3. Seats in a cinema	Seats arranged in numbered positions can be represented using array indexes.
4. Daily temperatures	Temperatures for each day of a week/month can be stored sequentially and accessed by day number.
5. Product prices	A list of prices can be stored in an array when the number of products is relatively stable and direct access is useful.

Advantages

- Fast access: An element can be accessed directly using its index, usuallyO(1).
- Simple: Easy to understand and implement.
- Memory efficient:Elements are stored contiguously in typical implementations.
- Good for iteration:Easy to process elements sequentially.

Disadvantages

- Fixed size: Traditional arrays cannot easily grow or shrink.
- Insertion/deletion can be slow:Adding or removing elements in the middle may require shifting elements, O(n).
- Wasted space: Allocating more space than needed can waste memory.
- Same-type elements: Traditional arrays generally store elements of the same data type.

2. Linked List

A linked listconsists of nodes where each node stores data and a link to another node.

Real-world example	Why a linked list is a natural fit
2. Browser history	Pages can be connected sequentially, allowing movement backward and forward through history.
3. Train carriages	Each carriage can conceptually link to the next carriage, making insertion/removal natural.
4. Image slideshow	Each image can point to the next image, allowing the slideshow to move through a sequence.
5. Undo/redo operations	Actions can be linked together so the system can move through previous or later states.

Advantages

- Dynamic size: Can grow or shrink as needed.
- Easy insertion/deletion: Adding or removing a node can be efficient when its position is known.
- No need for contiguous memory: Nodes can be stored in different memory locations.
- Useful for frequently changing data.

Disadvantages

- Slow random access: Finding the 100th element requires traversing earlier nodes, typically $O(n)$.
- Extra memory: Each node needs space for pointers/references.
- More complex: More difficult to implement than arrays.
- Poor cache performance: Nodes may not be stored next to each other in memory.

3. Stack

A stack follows **LIFO (Last In, First Out)** – the last item added is the first item removed.

Real-world example	Why a stack is a natural fit
1. Browser Back button	The most recently visited page is the first one you return t when pressing Back.
2. Undo feature	The most recent action is normally the first action that should be undone.
3. Stack of plates	The last plate placed on top is naturally the first plate removed.
4. Function calls	Programming languages use a call stack to track active

5. Expression evaluation	function calls: the most recent call finishes first

Advantages

- Simple structure:Only insertion and removal from the top are required.
- Fast operations:Push and pop are usually $O(1)$.
- Excellent for LIFO problems.
- Useful for recursion and function calls.

Disadvantages

- Limited access:You normally cannot directly access an item in the middle.
- Overflow:A fixed-size stack can run out of space.
- Not suitable for random access.
- Searching for a particular element can be $O(n)$.

4. Queue

A queue followsFIFO (First In, First Out)– the first item added is the first item removed.

Real-world example	Why a queue is a natural fit
1. Bank waiting line	The customer who arrives first is normally served first.
2. Printer queue	Documents can be printed in the order they were submitted.
3. Customer service requests	Support requests can be handled in the order they arrive.
4. CPU/process scheduling	Processes waiting for CPU time can be organized into a queue.
5. Online ticket booking requests	Requests can be processed in their arrival order when fair ordering is required.

Advantages

- Fair orderingFirst-come, first-served processing.
- Fast insertion/removalEnqueue and dequeue can be $O(1)$ with a suitable implementation.
- Excellent for scheduling.
- Useful for managing waiting tasks.

Disadvantages

- Limited access:Elements in the middle are not directly accessible.

- Searching is slow: Finding a particular element may require $O(n)$.
- Basic queues don't prioritize important items: A priority queue may be needed instead.
- Can become inefficient if implemented incorrectly using an ordinary array and repeated shifting.

5. Hash Table

A hash table stores data as key-value pairs, allowing very fast lookup using a key.

Real-world example	Why a hash table is a natural fit
1. Phone contacts	A person's name can be the key used to quickly find their phone number.
2. Student records	A student ID can be used as a key to quickly retrieve that student's information.
3. Product inventory	A product ID can quickly locate its price, quantity, and oth details.
4. Username lookup	A username can be used as a key to retrieve account information.
5. Dictionary	A word can be the key and its definition the associated value, making lookups very fast.

Advantages

- Very fast lookup: Average search, insertion, and deletion are approximately $O(1)$.
- Key-value organization: Naturally represents relationships between identifiers and data.
- Efficient for large datasets.
- Fast searching: No need to scan every element in normal cases.

Disadvantages

- Collisions: Different keys can produce the same hash value.
- No natural ordering: Data is generally not stored in sorted order.
- Extra memory: Hash tables can require additional storage.
- Worst-case performance: Operations can degrade to $O(n)$ if many collisions occur.

6. Tree

A tree organizes data in a hierarchical structure consisting of nodes connected by edges.

Real-world example	Why a tree is a natural fit
1. Computer file system	Folders contain files and subfolders, creating a natural hierarchy.
2. Company organization chart	A CEO can have managers, who have employees underneath them.
3. Family tree	Parents and children form a natural hierarchical relationship.
4. Website navigation menu	Main categories can contain subcategories and further nested pages.
5. Decision-making system	Each decision can lead to different branches and eventually to an outcome.

Advantages

- Excellent for hierarchical data.
- Efficient searching: Balanced search trees can provide $O(\log n)$ operations.
- Natural representation of parent-child relationships.
- Can efficiently support sorted data and range operations in appropriate tree types.

Disadvantages

- More complicated: More difficult to implement than arrays or lists.
- Unbalanced trees can become inefficient: A badly shaped binary search tree can degrade to $O(n)$.
- Extra memory: Nodes require references to children.
- Maintaining balance in structures such as AVL or red-black trees adds complexity.

QUESTION NO5

Advantages and Disadvantages of Data Structures

1. Array

Advantages:

- Easy to access elements using an index.
- Stores multiple elements of the same type.
- Fast access to any element.

Disadvantages:

- Fixed size in most implementations.
- Insertion and deletion can be time-consuming.
- Wastes memory if the allocated size is not fully used.

2. Stack
Advantages:

- Easy to implement and use.
- Follows LIFO (Last In, First Out) .
- Useful for function calls, undo operations, and expression evaluation.

Disadvantages:

- Can access only the top element directly.
- Stack size may be limited.
- Removing an element from an empty stack causes underflow .

3. Queue
Advantages:

- Follows FIFO (First In, First Out) .
- Useful for scheduling and managing tasks.
- Easy to add and remove elements from the appropriate ends.

Disadvantages:

- Direct access to middle elements is not possible.
- Searching for an element can be slow.
- A fixed-size queue can become full, causing overflow .